

Lab 2C: Advanced Skill Workflows - Test, Benchmark & Optimize

Duration: 35 min

After: Advanced Skill Systems lecture (Day 2, Block 3)

Prerequisites: Labs 2A-2B complete

Objective

Run a full **skill eval loop** (test set -> measure -> optimize -> retest) and an **A/B check** of your skill against raw Claude, then decide: keep, optimize, or retire.

Deliverable (toolkit chain 7 of 12)

An eval test set at `.claude/skills/code-review/tests/eval-prompts.md` plus a documented A/B verdict - the quality gate for your toolkit's skill.

START MARKER: `/clear done in business-dashboard.`

Fell behind? Run:

Set up this project with: (1) a code-review skill at `.claude/skills/code-review/SKILL.md` (frontmatter: name, description with review triggers, `disable-model-invocation: false`, `allowed-tools: Read, Grep, Glob`) producing RED/YELLOW/BLUE output, (2) a PostToolUse prettier hook and PreToolUse secret blocker registered in `.claude/settings.json`, (3) a `/review` command in `.claude/commands/`, (4) `CLAUDE.md` with project rules. Make scripts executable.

Exercise 1: Build the Eval Test Set (8 min)

Good skills trigger reliably. Prove it with data, not vibes.

Create an eval test set at `.claude/skills/code-review/tests/eval-prompts.md` with two sections:

SHOULD TRIGGER (true positives):

1. "Review the code in `src/routes/tasks.js`"
2. "Find bugs in this file"
3. "QA check on the task routes"
4. "Are there security issues in `server.js`?"
5. "Check this code for problems"

SHOULD NOT TRIGGER (true negatives):

1. "Write a new endpoint for user profiles"
2. "Explain how the Express router works"
3. "Run the tests"
4. "Refactor the dashboard for performance"
5. "Generate API documentation for `tasks.js`"

Exercise 2: Run the Eval (8 min)

/clear, then run at least 3 positives and 2 negatives. Score each:

Prompt	Fired? (RED/YELLOW/BLUE seen)	Pass ?
Review the code in src/routes/tasks.js	___	___
Find bugs in this file	___	___
QA check on the task routes	___	___
Write a new endpoint for user profiles	___ (should be NO)	___
Refactor the dashboard for performance	___ (should be NO)	___

Expected output marker: a score like 4/5. Perfect is rare - activation is probabilistic.

***Watch for a conflict:** if BOTH the skill and your /review command patterns respond to natural review prompts, make the command delegate: update .claude/commands/review.md body to "Use the code-review skill workflow to review \$ARGUMENTS." One source of truth.*

Exercise 3: A/B Check - Skill vs Raw Claude (10 min)

Does the skill actually improve output, or has the model caught up?

Step 1 - baseline (skill OFF):

```
Rename .claude/skills/code-review/SKILL.md to SKILL.md.bak
```

/clear, then:

```
Review src/routes/tasks.js for issues
```

Save the output (copy to a scratch file).

Step 2 - skill ON:

```
Rename .claude/skills/code-review/SKILL.md.bak back to SKILL.md
```

/clear, then the SAME prompt.

Step 3 - compare:

Criteria	Without skill	With skill
Structured RED/YELLOW/BLUE output?		
Used the project checklist?		
Cited project-specific conventions?		
Consistent format on a rerun?		

Verdict:

Result	Action
Skill wins clearly	Keep
Skill wins slightly	Keep, retest after model updates

Result	Action
Raw Claude just as good	Consider retiring - the model caught up

Two skill types: capability uplift (teaches something new - keeps its edge) vs workflow/preference (enforces YOUR format - models catch up). Yours is a workflow skill: its value is consistency, not capability.

Exercise 4: Optimize the Description (6 min)

Feed real failures back into the description:

```
Read my code-review skill at .claude/skills/code-review/SKILL.md.
Based on these eval results: [paste your table - which fired, which shouldn't have]
Suggest an improved description: sharper trigger phrases, tighter negative boundaries,
clearer separation from documentation and refactoring tasks. Update only the description -
keep the workflow unchanged.
```

`/clear`, rerun the failed prompts.

Expected output marker: previously-failing prompts now behave. This evaluate -> optimize -> retest cycle is exactly how Anthropic tunes its own bundled skills.

Exercise 5: Hygiene Check (3 min)

```
/doctor
```

Expected output marker: the July 2026 `/doctor` flags **unused skills**, unused MCP servers, and slow hooks. If `ship-checklist` (Lab 2A) shows as unused - that's fine, it's manual-only. If `code-review` shows unused, your eval and doctor disagree: trust the eval, but note the drift.

FINISH MARKER - Success Checklist

- [] Eval test set exists with 5 positives + 5 negatives
- [] At least 5 prompts scored; trigger rate recorded
- [] A/B run completed with the comparison table filled in
- [] Keep/optimize/retire verdict written down with a reason
- [] Description optimized and failed prompts retested
- [] `/doctor` run; unused-skill flags reviewed

If Stuck

Problem	Recovery
Nothing triggers after rename	You forgot to <code>/clear</code> after restoring SKILL.md
Both command and skill fire	Make <code>/review</code> delegate to the skill (Exercise 2 note)
A/B outputs look identical	Your rules match model defaults - add project-specific checks the model can't guess
Eval results vary run to run	Normal (probabilistic); score across 2 runs and average

Debrief Questions

1. Why rerun the eval set after every model update?
2. Capability uplift vs workflow skill - which survives model progress, and which is yours?
3. What signal would make you retire a skill rather than optimize it?